RONDA

Guía Técnica de Setup: Integración WOMPI

Colombia 2026 | Paso a Paso

0. RESUMEN EJECUTIVO TÉCNICO

Objetivo: Configurar WOMPI como proveedor de pagos primario para RONDA en Colombia. Los usuarios pagan en pesos colombianos (COP) vía tarjeta de crédito, PSE o transferencia bancaria. El dinero llega a la cuenta bancaria de RONDA en T+1.

Timeline Total: 5-7 días (desde registro hasta primera transacción real)

Fase	Duración	Owner	Prerequisitos
Fase 1: Registro	5 minutos	César	Email + datos básicos
Fase 2: KYC	1-2 horas	César	Cédula + selfie + cuenta bancaria
Fase 3: API Setup	2-3 horas	Dev / César	API keys + credenciales
Fase 4: Testing	1-2 días	QA / Dev	Sandbox transactions
Fase 5: Go Live	1 día	Dev / Ops	Modo producción habilitado

1. FASE 1: REGISTRO EN WOMPI (5 Minutos)

Paso 1.1: Acceder a wompi.co

Abre tu navegador y ve a **https://dashboard.wompi.co/register**

Campos a completar:

- Email comercial (recomendado: pagos@ronda.co)
- Nombre del Negocio: "RONDA Speed Dating"
- Teléfono: +57 (número celular actual)
- Contraseña: Fuerte (min 12 caracteres, mayúsc + número + símbolo)

> ✅ **Recomendación:** Usa Gmail o Outlook para la cuenta. NO uses email de godaddy/hosting; puede bloquearse.

Paso 1.2: Confirmar Email

WOMPI enviará email de confirmación a tu bandeja. Haz clic en "Confirmar Email" dentro de los próximos 24 horas.

Una vez confirmado, se abre el dashboard de WOMPI.

> 📧 **Email No Llega?** Revisa spam. Si sigue sin llegar, contacta a soporte@wompi.co con tu email + teléfono.

2. FASE 2: VERIFICACIÓN KYC (1-2 Horas)

Paso 2.1: Datos Personales

En el dashboard, ve a **Configuración → Información de la Cuenta**

Campos obligatorios:

- **Tipo de Documento:** Cédula de Ciudadanía (CC) o NIT si es empresa
- **Número de Documento:** Tu cédula / NIT (ej: 1234567890)
- **Nombres y Apellidos:** Exacto como aparece en cédula
- **Fecha de Nacimiento:** DD/MM/YYYY
- **País de Residencia:** Colombia
- **Ciudad:** Bogotá, D.C.
- **Dirección:** Calle 82 #10-50 Apto 1201, Bogotá (ejemplo)

Paso 2.2: Documento de Identidad (Upload)

Descarga y prepara:

- **Foto 1:** Frente de cédula (bien iluminada, legible, sin reflejos)
- **Foto 2:** Reverso de cédula
- **Selfie:** Tu foto con cédula en mano (cara clara, sin gafas de sol)

Requerimientos técnicos:

- Formato: JPG o PNG
- Tamaño: 5MB máximo por archivo
- Resolución: 1024x768 mínimo
- NO usar documentos escaneados mal; usa celular directo

> ⚠ **QA Automática:** Si WOMPI rechaza tus fotos por "calidad", revisa:
> - ¿La cédula ocupa 70%+ de la foto?
> - ¿Está completamente dentro del frame (sin cortes)?
> - ¿Está bien iluminada (sin sombras)?
> - ¿El selfie muestra claramente tu cara Y la cédula?

Paso 2.3: Información Bancaria

Vincula la cuenta donde quieres recibir los pagos:

Datos a tener listos:

- Banco: (ej: Bancolombia, BBVA, Davivienda, Bogotá)
- Tipo de Cuenta: Ahorros o Corriente

- Número de Cuenta: 16-18 dígitos sin espacios
- Titular: Debe coincidir con tu nombre en WOMPI

> ℹ **Bancos Soportados:** Todos los bancos colombianos son válidos. WOMPI transfiere via ACH colombiano en T+1.

Paso 2.4: Validación Automática

Una vez envíes los documentos, WOMPI hace validación automática:

- **T+0 (Ahora):** Envías fotos y datos
- **T+30 min:** Validación automática (OCR + liveness check)
- **T+1 a T+4 horas:** Review manual si necesario
- **T+1 a T+2 días:** Aprobación y activación de comercio

> ✅ **Cuándo estás listo:** Recibirás email "Tu cuenta ha sido aprobada". Desde ese momento puedes hacer transacciones en Sandbox.

---

3. FASE 3: API SETUP (2-3 Horas)

Paso 3.1: Obtener API Keys

Una vez aprobado, ve a **Desarrollador → API Keys** en dashboard.wompi.co

Copias y guardas en archivo seguro:

- **WOMPI_SANDBOX_PUBLIC_KEY:** pk_test_xxxxxxxx...
- **WOMPI_SANDBOX_PRIVATE_KEY:** prv_test_xxxxxxxx...

> ⚠ **CRÍTICO:** Nunca compartas la PRIVATE_KEY. Guárdala en archivo .env (no en Git). Estructura recomendada:

```
# .env (gitignore + encrypted) WOMPI_SANDBOX_PUBLIC=pk_test_xxxxxxxx WOMPI_SANDBOX_PRIVATE=prv_test_xxxxxxxx
WOMPI_SANDBOX_INTEGRITY_KEY=xxxxx WOMPI_PROD_PUBLIC=pk_prod_xxxxxxxx (después) WOMPI_PROD_PRIVATE=prv_prod_xxxxxxxx (después)
```

Paso 3.2: Integración Técnica

Dependiendo de tu stack técnico:

Opción A: Typeform + Zapier (Sin Dev Requerido)

Si usas Typeform para signup, configura así:

- Typeform form → Zapier → WOMPI API
- Zapier crea automáticamente transacción en WOMPI
- Usuario recibe link de pago vía WhatsApp/Email

Setup Zapier:

1. Crea nuevo Zap: Typeform (trigger) → Webhooks by Zapier (action)
2. Configura URL webhook en WOMPI para recibir confirmaciones
3. Mapea campos: Typeform → WOMPI (nombre, email, monto, evento_id)

```
// Webhook Body en Zapier (ejemplo): { "amount_in_cents": 5700000, // 57.000 COP "currency": "COP", "customer_email":
"usuario@example.com", "customer_phone_number": "+57XXXXXXXXX", "reference": "EVENTO_20260823_001", "description": "Entrada RONDA -
Martes Zona G", "redirect_url": "https://tuapp.com/success" }
```

Opción B: Bubble/Low-Code (Recomendado para RONDA)

Si usas Bubble como tu app, integración nativa:

1. Ve a Bubble → Plugins → Busca "WOMPI"
2. Instala plugin oficial WOMPI (si existe) o usa API plugin genérico
3. Configura tus API keys en plugin settings
4. Crea workflow: "User clicks Pagar" → WOMPI Create Transaction

```
// Bubble Workflow (pseudocódigo): Trigger: "Pagar" button clicked Action: WOMPI API Call Method: POST
/merchants/{merchant_id}/transactions Body: { "amount_in_cents": Current_Event.price_cents, "currency": "COP", "customer_email":
```

```
Current_User.email, "reference": Current_Event.id, "redirect_url": "https://tuapp.bubbleapps.io/success" } Response: Store transaction_id → Redirect to WOMPI checkout
```

Opción C: React/Node Custom (Para Dev)

Si tienes dev interno, setup estándar:

```
// Node.js Backend (Express) const axios = require('axios'); const WOMPI_API = 'https://api.wompi.co/v1'; const WOMPI_PRIVATE_KEY = process.env.WOMPI_SANDBOX_PRIVATE; async function createTransaction(data) { try { const response = await axios.post( `${WOMPI_API}/merchants/default/transactions`, { amount_in_cents: data.amountInCents, // 5700000 = 57.000 COP currency: 'COP', customer_email: data.email, customer_phone_number: data.phone, reference: data.eventId, redirect_url: 'https://tuapp.com/success' }, { headers: { 'Authorization': `Bearer ${WOMPI_PRIVATE_KEY}`, 'Content-Type': 'application/json' } } ); return response.data.data; } catch (error) { console.error('WOMPI Error:', error.response.data); throw error; } } // Frontend (React) async function handlePayment(eventData) { const txn = await createTransaction({ amountInCents: eventData.price * 100, email: currentUser.email, phone: currentUser.phone, eventId: eventData.id }); // Redirect to WOMPI checkout window.location.href = txn.processing_data.checkout_url; }
```

4. FASE 4: TESTING EN SANDBOX (1-2 Días)

Paso 4.1: Tarjetas de Prueba

WOMPI proporciona tarjetas virtuales para testing:

Escenario	Número Tarjeta	Vencimiento	CVV	Resultado
Pago Exitoso	4242 4242 4242 4242	12/25	123	✅ Aprobado
Tarjeta Rechazada	4000 0000 0000 0002	12/25	123	❌ Rechazado
Fondos Insuficientes	4000 0000 0000 9995	12/25	123	❌ Fondos insuf.
3D Secure	4053 9010 1000 0009	12/25	123	✅ Con OTP

ℹ **Nota:** En Sandbox, cualquier OTP (One-Time Password) funciona. Usa "123456" o lo que pida WOMPI.

Paso 4.2: Test Cases

Ejecuta estos tests antes de go-live:

Test ID	Acción	Resultado Esperado	✅ OK?
TEST-001	Crear transacción con 57.000 COP (tarjeta válida)	Status: APPROVED + redirect a checkout	[ ]
TEST-002	Crear transacción con tarjeta rechazada	Status: DECLINED + error message	[ ]
TEST-003	Webhook de confirmación recibido	Event status = TRANSACTION.COMPLETED en tu servidor	[ ]
TEST-004	Reporte en dashboard WOMPI	Transacción visible en Historial	[ ]
TEST-005	Email de confirmación al usuario	Usuario recibe "Pago confirmado" + comprobante	[ ]
TEST-006	PSE (transferencia bancaria)	Opción disponible en checkout	[ ]

✅ **Cuando todos los tests pasen:** Estás listo para Producción.

5. FASE 5: SWITCH A PRODUCCIÓN (1 Día)

Paso 5.1: Obtener Prod Keys

En dashboard.wompi.co, ve a **Configuración → API Keys**

Descarga:

- **WOMPI_PROD_PUBLIC_KEY:** pk_prod_xxxxxxxx...
- **WOMPI_PROD_PRIVATE_KEY:** prv_prod_xxxxxxxx...
- **WOMPI_PROD_INTEGRITY_KEY:** (para verificación de webhooks)

⚠ **Almacenamiento Seguro:** Guarda estas keys en variables de entorno (production-only). NO en Git, NO en logs.

Paso 5.2: Configurar Environment

En tu servidor/hosting, configura environment variables:

```
# Producción (production.env o similar) NODE_ENV=production WOMPI_ENV=production WOMPI_PUBLIC_KEY=pk_prod_xxxxxxxx
WOMPI_PRIVATE_KEY=prv_prod_xxxxxxxx WOMPI_INTEGRITY_KEY=int_prod_xxxxxxxx WOMPI_REDIRECT_URL=https://ronda.co/success
WOMPI_WEBHOOK_URL=https://ronda.co/api/webhooks/wompi
```

ℹ **Dónde configurar esto:**

- **Heroku:** Settings → Config Vars
- **Vercel:** Settings → Environment Variables
- **AWS Lambda:** Environment variables en console
- **Bubble:** Settings → Private keys

Paso 5.3: Habilitar Modo Producción

En tu código, cambia:

```
// Antes (Sandbox): const WOMPI_API = 'https://api.wompi.co/v1'; const API_KEY = process.env.WOMPI_SANDBOX_PRIVATE; // Después
(Producción): const WOMPI_API = 'https://api.wompi.co/v1'; // Mismo endpoint const API_KEY = process.env.WOMPI_PROD_PRIVATE; // Key
diferente // O mejor aún: const WOMPI_API = 'https://api.wompi.co/v1'; const API_KEY = process.env.NODE_ENV === 'production' ?
process.env.WOMPI_PROD_PRIVATE : process.env.WOMPI_SANDBOX_PRIVATE;
```

Paso 5.4: Primer Pago Real

Ejecuta primer pago real con tarjeta de débito/crédito actual:

1. Usuario accede a app (producción)
2. Selecciona evento y hace click en "Pagar"
3. Ingresa datos reales de tarjeta (57.000 COP)
4. Recibe confirmación en app + email
5. Dinero se procesa en WOMPI (aparece en T+1 en tu banco)

> ✅ **Validación:**
> - ¿Transacción aparece en dashboard WOMPI?
> - ¿Email de confirmación recibido por usuario?
> - ¿Dinero llega a tu cuenta bancaria en T+1?
>
> Si los 3 sí → ¡Go-live exitoso!

---

6. WEBHOOKS Y SEGURIDAD

Paso 6.1: Configurar Webhooks

Para que WOMPI te notifique cuando ocurre un pago, configura webhook:

En dashboard.wompi.co → **Desarrollador → Webhooks**

Registra estos eventos:

- ✅ transaction.completed (pago exitoso)
- ✅ transaction.failed (pago fallido)
- ✅ transaction.declined (tarjeta rechazada)

URL Endpoint: https://tuapp.com/api/webhooks/wompi

> ℹ **El endpoint debe:**
> - Recibir POST request
> - Validar firma (WOMPI_INTEGRITY_KEY)
> - Actualizar estado de evento en tu BD
> - Retornar 200 OK

Paso 6.2: Validar Webhook (Seguridad Crítica)

SIEMPRE valida que webhook viene de WOMPI (no es fake):

```
// Node.js - Validar webhook signature const crypto = require('crypto'); function validateWompiWebhook(req) { const signature = req.headers['x-wompi-signature']; const timestamp = req.headers['x-wompi-timestamp']; const integrityKey = process.env.WOMPI_INTEGRITY_KEY; // Construir string para validación const bodyString = JSON.stringify(req.body); const message = `${timestamp}.${bodyString}`; // Calcular HMAC const expectedSignature = crypto .createHmac('sha256', integrityKey) .update(message) .digest('hex'); // Validar if (signature === expectedSignature) { return true; // ✅ Webhook válido } else { console.error('Invalid webhook signature'); return false; // ❌ Fake/alterado } } // Express middleware app.post('/api/webhooks/wompi', (req, res) => { if (!validateWompiWebhook(req)) { return res.status(403).json({ error: 'Invalid signature' }); } // Procesar evento const event = req.body.event; const data = req.body.data; if (event === 'transaction.completed') { updateEventStatus(data.reference, 'PAID'); } res.json({ received: true }); });
```

> ⚠ **CRÍTICO:** Sin validación de firma, alguien podría falsificar webhooks y marcar pagos como completados sin que lo sean. Siempre valida.

7. MANTENIMIENTO Y MONITOREO

Paso 7.1: Dashboards Diarios

Revisa cada día (primeras 2 semanas):

Dashboard	Qué Revisar	Frecuencia
WOMPI Dashboard	Transacciones procesadas, rechazadas, monto total	Diario (primeras 2 sem)
Tu App Analytics	% conversión (usuarios que llegan a checkout)	Diario
Email Transaccionales	¿Usuarios reciben confirmación? ¿Sin spam?	Spot checks
Banco (Tu Cuenta)	¿Dinero llega en T+1 como esperado?	T+1 después de cada transacción

Paso 7.2: Alerts y Escalación

Configura alertas para problemas:

Evento	Acción
Transacción fallida (DECLINED)	Email a usuario + link reintentar
Webhook no recibido por 5+ min	Alert a Slack/Email a dev
Tasa de rechazo sube a >10%	Revisar WOMPI dashboard + contactar support
Dinero no llega a banco (T+2)	Contactar WOMPI soporte inmediatamente

Paso 7.3: Troubleshooting Común

Problema: "Transacción stuck en PENDING"

Solución: Webhook puede no haber sido recibido. Revisa logs del servidor + reinicia webhook desde WOMPI dashboard.

Problema: "Usuario no recibe email de confirmación"

Solución: Revisa WOMPI webhook payload → si transaction.completed llegó, el problema es tu email service (Brevo/SendGrid). Revisa spam.

Problema: "Dinero no aparece en banco después de 2 días"

Solución: Contacta WOMPI soporte con transaction_id. Probable: cuenta bancaria incorrecta o rechazo ACH.

8. REFERENCIAS Y RECURSOS

Documentación Oficial

- **WOMPI Docs API:** https://docs.wompi.co/
- **WOMPI Dashboard:** https://dashboard.wompi.co
- **WOMPI Soporte Email:** soporte@wompi.co
- **WOMPI Soporte Chat:** En dashboard → Help
- **Status Page:** https://status.wompi.co/

Contactos de Soporte

WOMPI Soporte (Colombia):

- Email: soporte@wompi.co
- Teléfono: +57 (1) 7867-010 ext. 2
- Chat en dashboard (24/7)

Para Problemas Críticos (Produción): Usa chat en dashboard → escalará a ingeniero en <5 min.

Checklists Finales

✓ Pre-Go-Live Checklist:

- [ ] Registro WOMPI completado
- [ ] KYC aprobado
- [ ] API keys en .env (sandbox)
- [ ] Integración técnica testing
- [ ] 6+ transacciones de prueba exitosas
- [ ] Webhooks recibiendo eventos
- [ ] Emails de confirmación funcionando
- [ ] Prod API keys configuradas
- [ ] URL webhook en producción registrado
- [ ] Firma webhook siendo validada
- [ ] Primera transacción real procesada ✓
- [ ] Dinero recibido en banco ✓

✓ Post-Go-Live (Primeras 2 Semanas):

- [ ] Revisar dashboard diario
- [ ] Monitorear tasa de rechazo
- [ ] Validar llegada de dinero a banco
- [ ] Recopilar feedback de usuarios sobre checkout

- [ ] Documentar cualquier issue para escalación
